



Programme

Diploma in Software Development – 240 credits

Year Two

Course

**SD202 Cross-Platform Development
(Level 6, 15 Credits, Version 2)**

Assessment 1

Written Assignment

Weighting within course:

25%

Objective:

This course aims to students with further understanding of academic and professional investigation and presentation approaches, in order to creatively ideate and apply critical thinking to the production and presentation of well-researched and persuasive proposals for software products.

The objective of this assessment is to test your understanding of how your personal expertise can used to contribute to the execution of a team-based software project.

Learning Outcomes (LOs) covered:

LO 3: Demonstrate understanding of how development in an area of personal expertise can used to contribute to the execution of a team-based software project.

Qualification Graduate Profile Outcomes (GPOs) covered:

GPO 1: Analyse requirements and apply appropriate modelling tools, SDLC and HCI principles, to design and document software solutions for a range of problems in an organisational context, including creating accurate and clear technical and user documentation/resources

GPO 2: Write and maintain complex programs using design patterns, data structures and algorithms to meet specifications and software development standards.

GPO 5: Integrate security and privacy principles throughout software development to ensure application security and system integrity using current best practices.

GPO 10: Apply project management tools and techniques, using critical thinking, enterprise skills and knowledge of project planning, management and control, to an IT related project, to analyse and solve problems.

Assessment Matrix:

Learning Outcome	GPO (DSD)	Assessment Task	Weightage
LO 3: Demonstrate understanding of how development in an area of personal expertise can used to contribute to the execution of a team-based software project.	GPO 1, GPO 2, GPO 5 and GPO 10	Task 1	25
		Task 2	25
		Task 3	25
		Task 4	25
		Total	100

Grading:

The final grade will be determined by the score achieved in this assessment based on the following table. Should a second or third attempt be required the maximum contribution toward the overall mark for the tasks that required a second or third assessment attempt is 50%. A late submission is considered a second attempt, so the contribution will be capped at 50%.

Grade	Range
A+	Meet all course requirements, range (90-100%)
A	Meet all course requirements, range (85-89%)
A-	Meet all course requirements, range (80-84%)
B+	Meet all course requirements, range (75-79%)
B	Meet all course requirements, range (70-74%)
B-	Meet all course requirements, range (65-69%)
C+	Meet all course requirements, range (60-64%)
C	Meet all course requirements, range (55-59%)
C-	Meet all course requirements, range (50-54%)
D	Did not meet all course requirements, range (40-49%)
E	Did not meet all course requirements, mark range (0-39%)

Candidate's Assessment Instructions:

- This assessment is an **open-book activity**, you can use your own course and review notes as well as offline or online resources, such as textbooks or online journals.
- This assessment is an individual assessment, and you are required to complete it on your own.
- You can always ask your online tutor if you need further explanation if the instructions are unclear.
- Your work should not be plagiarised. Plagiarism includes copying material without acknowledging it, copying from another student, getting another person to help you with your assessment, using material from commercial essays or assignment services, or using AI to create the answers.
- The purpose of this assessment is to assess your knowledge. In the event YooBee suspects collusion, this will be addressed. For more information on plagiarism, please refer to the Student Handbook.
- Submit your completed assessment online in the correct space provided.
- Marks and feedback will be returned within 15 days of the submission date.
- By completing and submitting an assessment you are authenticating that the work is original and does not violate plagiarism or copyright law. Authenticity is checked where any breaches of academic integrity are suspected. Please refer to the Student Handbook for further information.

Submission Instructions:

Please submit the following to your LMS:

- An assessment document constituting all tasks of the assessment.

Assessment Tasks Completion Timeline:

Week	What needs to be done	What needs to be submitted
Week1	- Assessments 1 and 2 to be released. - Student starts working on Assessment 1	
Week 2	- Student keeps working on Assessment 1	
Week 3	- Recommend completing Assessment 1 by the end of the week	
Week 4	- Students for a group. Students start working on Assessment 2	Submit via LMS: - Assessment 1 document
Week 5	- The group continues working on Assessment 2	
Week 6	- The group continues working on Assessment 2	
Week 7	- The group continues working on Assessment 2	
Week 8	- Student to complete Assessment 2 with individual reflection	Submit via LMS: - Your team project proposal, including all team member names, student IDs, and course codes. - Presentation file - Individual reflection report from every team member

ASSESSMENT

LO3: Demonstrate understanding of how development in an area of personal expertise can be used to contribute to the execution of a team-based software project.

Description

In this assignment, students should implement a solution that adheres to logical problem-solving methods and effective communication of the code and its functionality. Students will write code in the LMS coding sandbox environment, where their solutions will be tested against predefined test cases. By attempting this assignment, students' skills will be evaluated in a more hands-on oriented approach.

OBJECTIVES OF THE ASSIGNMENT

- Integrate academic practice and research methods to identify and investigate a problem in a professional context.
- Generate effective communication to share ideas and concepts and enhance professional practice.
- Apply Python programming skills to develop a functional solution to a real-world problem.
- Utilise an online coding environment to develop, test, and refine Python code.

STEPS INVOLVED

1. Problem Identification:
 - Understand the given problem by applying divide-and-conquer or other problem-solving techniques.
2. Research and Planning:
 - Conduct brief research to understand the problem. Outline the approach to solving it logically.
 - Develop a simple plan or pseudocode for the solution.
3. Coding in LMS Sandbox:
 - Access the LMS coding playground.
 - Write Python code to solve the chosen problem. Ensure the code follows best practices and is well-documented with comments explaining the logic.
4. Testing and Debugging:
 - Run the code in the sandbox environment.
 - The code will be tested against predefined test cases to ensure it produces the correct output.
 - If any test cases fail, debug and refine the code accordingly. Students can submit multiple times until all test cases pass. (Attempts will be tracked)
5. Submission:
 - Once the code passes all test cases, submit the final code through LMS.

Task 1: Detective (25%)

As a homicide detective, you have been tasked with investigating a case implicating three suspects: John, Cathy, and Samuel, who are under suspicion for murder. The investigation involves considering five potential murder weapons and five different rooms where the crime could have occurred. Initial findings suggest various plausible scenarios based on the combination of suspects, rooms, and tools:

Situations:

- 1- John, Bathroom, Lead pipe
- 2- Cathy, Dining, Revolver
- 3- Cathy Ball, Dagger
- 4- Samuel, Master bedroom, Revolver
- 5- John, kitchen, dagger
- 6- Cathy, Master bedroom, Hammer

Each tool and room combination are assigned a specific value, detailed as follows:

Room (Value)	Tool (Value)
Ball (0.5)	Dagger (0.4)
Dining (0.9)	Revolver (0.7)
Kitchen (0.1)	Lead pipe (0.1)
Master bedroom (0.3)	Hammer (0.8)
Bathroom (0.3)	

The goal is to prioritize the scenarios based on their total value, which indicates the likelihood of the crime. For instance, in situation 1 described earlier, the total value would be computed as $0.3 + 0.1 = 0.4$. Develop a code to accomplish this objective. It's essential to avoid hard coding the situation values. Instead, devise a method to extract the pertinent scores corresponding to the scenario at hand and subsequently arrange the scenarios in descending order.

Task 2: Sports Emojis (25%)

You are tasked with developing a Python code to convert a list of keywords entered by a user into corresponding emojis. The user must input five keywords separated by commas, each of which will be transformed into an emoji. The selection of emojis is at your discretion, ensuring a one-to-one mapping between keywords and emojis. Here is the list of keywords to choose from:

Keywords: Soccer, Cricket, Rugby, Golf, Volleyball, Hockey, Squash, Bowling, Table-tennis, Handball, Basketball, Boxing

In case the user inputs a keyword not found in the provided list, it should be represented by a confusion emoji. Furthermore, if the number of keywords entered falls below five, an error prompt should appear, requesting the user to re-enter the keywords. Additionally, duplicate keywords are not permitted, and if any duplicates are detected, an error message should be displayed, prompting the user to re-enter the keywords. Below are sample inputs and their corresponding expected outputs for reference.

Scenarios:

Input: Bowling,Soccer,Cricket,Rugby,Hockey

Output: 🏏⚽🏏🏏🏏

Input: Table-tennis,Bowling,Golf,Netball,Boxing

Output: 🏓🤖🏌️🤖🥊

Input: Volleyball,Squash,Boxing

Output: Error. Number of keywords is less than 5. Re-enter keywords
(Then you need to prompt again for input)

Input: Soccer,Hockey,Handball,Boxing,Hockey

Output: Error. Repeating keywords are not allowed. Re-enter keywords
(Then you need to prompt again for input)

Task 3: File Formatting (25%)

As an experienced data engineer, you've been tasked with refining an existing data file recorded by an inexperienced data engineer. The initial version of this document entailed records of five BSE/DSD Year 2 students at Yoobee College. Regrettably, the data formatting in this document lacked structure, where:

- 1- The first 5 lines represented the student IDs
- 2- The second 5 lines represented their respected first names
- 3- The third 5 lines represented their respected last names
- 4- The fourth 5 lines represented their respected campuses
- 5- The fifth 5 lines represented their respected mode of study

Your objective is to create another text document (student_record_v2.txt) by initially parsing this document and restructuring the data into a more organized format. An exemplar initial version of the record is stored in the file student_record_v1.txt, while the formatted document you are to produce is provided in student_record_v2.txt.

Note: In the formatted document, an additional set of records, i.e., emails, are incorporated, represented as follows:

firstname_lastname@yoobee colleges.com

Develop a Python program that performs the above task.

Task 4: Code Analysis (25%)

Analyse the provided code in detail explaining the following:

- 1- The purpose of the code
- 2- The functionality of each function
- 3- The kind of error generated by the code providing some insight on the error and why this happens.
- 4- If we replace the code **squared_diff_sum += (s1[i] - s2[i]) ** 2** in the **distance_fn()** by **squared_diff_sum += (s1[i] - s2[i])**, what difference can you see in the output. Comment with justification.

Code:

```

def sequenece_checker(s1,s2):
    if len(s1) != len(s2):
        return False
    return True

def calculate_mean(anysequence):
    total = 0
    count = 0
    for num in anysequence:
        total += num
        count += 1
    return total / count

def distance_fn(s1, s2):
    squared_diff_sum = 0
    for i in range(len(s1)):
        squared_diff_sum += (s1[i] - s2[i]) ** 2
    return squared_diff_sum

# Example
s1 = [1, 2, 4, 5, 9]
s2 = [6, 7, 8, 9, 10]

cond = sequenece_checker(s1,s2)
if (cond):
    me1 = calculate_mean(s1)
    me2 = calculate_mean(s2)
    distance = distance_fn(s1, s2)
    print("Mean of sequence 1:", me1)
    print("Mean of sequence 2:", me2)
    print("Distance between the sequences:", distance)
else:
    print('Warning! Error.')

```

Marking Rubrics

Criterion	Weightage	Evidence				
Task		A+, A, A-	B+, B, B-	C+, C, C-	D	E
LO 3 (GPO 1, GPO 2, GPO 5, and GPO 10)						
Task 1: Detective	25%	The student has developed the code to accomplish the required objective. They have avoided hard coding the situation values. Also, they have devised a method to extract the pertinent scores corresponding to the scenario at hand and subsequently arranged the scenarios in descending order. The evidence provided is excellent.	The student has developed the code to accomplish the required objective. They have avoided hard coding the situation values. Also, they have devised a method to extract the pertinent scores corresponding to the scenario at hand and subsequently arranged the scenarios in descending order. However, there are some errors in the implementation. The evidence provided is adequate.	The student has developed the code to accomplish the required objective. They have avoided hard coding the situation values. However, they haven't devised a method to extract the pertinent scores corresponding to the scenario at hand and subsequently arranged the scenarios in descending order. The evidence provided is satisfactory.	The student has poorly developed the code to accomplish the required objective.	The student has attempted the task but has provided mostly irrelevant content. OR The student has not attempted this task.
Task 2: Sports Emojis	25%	The student has developed a Python code to convert all five keywords entered by a user into corresponding emojis. The evidence provided is excellent.	The student has developed a Python code to convert three to four keywords entered by a user into corresponding emojis.	The student has developed a Python code to convert two keywords entered by a user into corresponding emojis.	The student has developed a Python code to convert only one keyword entered by a user into corresponding emojis.	The student has attempted the task but has provided mostly irrelevant content. OR The student has not attempted this task.
Task 3: File Formatting	25%	The student has created a text document by initially parsing the document and restructuring the data into a required organized format in Python. The evidence provided is excellent.	The student has created a text document by initially parsing the document and restructuring the data into a required organized format in Python. The text document has some errors and doesn't fulfill all requirements.	The student has created a text document by initially parsing the document and restructuring the data into a required organized format in Python. The text document has several errors and doesn't fulfill most requirements.	The student has attempted the task but has provided mostly irrelevant content.	The student has not attempted this task.
Task 4: Code Analysis	25%	The student has analysed the provided code in detail explaining all four points. The analysis is excellent.	The student has analysed the provided code in detail explaining three points. The analysis is good.	The student has analysed the provided code in detail explaining two points. The analysis is satisfactory.	The student has analysed the provided code in detail explaining only one point.	The student has attempted the task but has provided mostly irrelevant content. OR The student has not attempted this task.
Sum	100%					

Notes: To pass this assessment, you must meet all the requirements of each of the learning outcomes (irrespective of the numerical grade awarded) and achieve **PASS** results for each learning outcome.